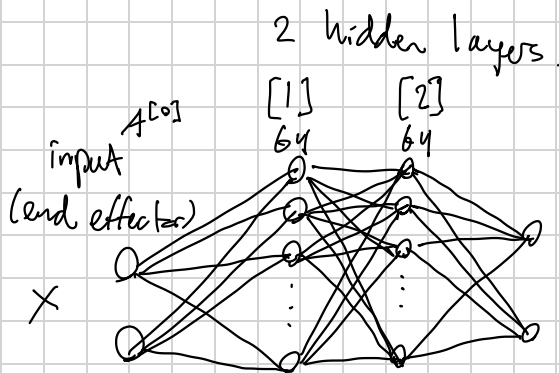




Hidden size 64 is kinda arbitrary. Has to be large enough, not too large.

$$A^{[0]} = X = \begin{bmatrix} x_1 & x_2 & x_3 & \dots & x_m \\ y_1 & y_2 & y_3 & \dots & y_m \end{bmatrix},$$

$$\begin{aligned} A^{[0]} &\in \mathbb{R}^{m \times 2} \\ b^{[1]} &\in \mathbb{R}^{64 \times 1} \\ W^{[1]} &\in \mathbb{R}^{64 \times 2} \\ z^{[1]} &\in \mathbb{R}^{m \times 64} \end{aligned}$$

$$z^{[1]} = A^{[0]} (W^{[1]})^T + b^{[1]}$$

Same as $z^{[1]} = W^{[1]} A^{[0]} + b^{[1]}$
but changed due to

PyTorch notation.

$$H^{[1]} = \text{ReLU}(z^{[1]})$$

Would be a linear regression if not for ReLU

$$\text{ReLU}(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases}$$



Rectified Linear Unit.

$$z^{[2]} = H^{[1]} (W^{[2]})^T + b^{[2]}$$

$$H^{[2]} = \text{ReLU}(z^{[2]})$$

$$\hat{y} = H^{[2]} (W^{[3]})^T + b^{[3]}, \quad \hat{y} \in \mathbb{R}^{m \times 2}$$

Each row contains predicted joint angle: $\hat{y}_i = [\hat{\theta}_{1,i}, \hat{\theta}_{2,i}]$

True target value: $y_i = [\theta_{1,i}, \theta_{2,i}]$

Goal is $\hat{y} \approx y$. We use mean squared error to measure error.

$$L = \text{MSE}(\hat{y}, y). \text{ For one example: } L = (\hat{a}_1 - a_1)^2 + (\hat{a}_2 - a_2)^2$$

m examples:

$$L = \frac{1}{m} \sum_{i=1}^m [(\hat{a}_{1,i} - a_{1,i})^2 + (\hat{a}_{2,i} - a_{2,i})^2]$$

$\frac{\partial L}{\partial \hat{y}}$ means: How does the loss change if we slightly change network output.

$$L = (\hat{a}_1 - a_1)^2 + (\hat{a}_2 - a_2)^2, \quad \frac{\partial L}{\partial \hat{a}_1} = 2(\hat{a}_1 - a_1), \quad \frac{\partial L}{\partial \hat{a}_2} = 2(\hat{a}_2 - a_2)$$

$$\Rightarrow \frac{\partial L}{\partial \hat{y}} = \begin{bmatrix} 2(\hat{a}_1 - a_1) \\ 2(\hat{a}_2 - a_2) \end{bmatrix} = 2(\hat{y} - y)$$

Backwards pass starts by differentiating the loss with respect to the network output

$$d\hat{y} = \frac{\partial L}{\partial \hat{y}}$$

$$\text{For mean squared error: } d\hat{y} = \frac{2}{m}(\hat{y} - y)$$

$$\text{Since final layer is } \hat{y} = H^{[2]}(W^{[3]})^T + b^{[3]}$$

Back propagation starts from the output error and moves backward through the network. Computes how much each weight and bias contributed to the loss.

$$\frac{\partial L}{\partial W^{[3]}}, \frac{\partial L}{\partial b^{[3]}}, \frac{\partial L}{\partial W^{[2]}}, \frac{\partial L}{\partial b^{[2]}}, \frac{\partial L}{\partial W^{[1]}}, \frac{\partial L}{\partial b^{[1]}}$$

then the optimizer updates parameters

$$W := W - \alpha \frac{\partial L}{\partial W}, \quad b := b - \alpha \frac{\partial L}{\partial b}, \quad \alpha \text{ is learning rate.}$$

↑ ↑
This allows us to find local minima for loss function, allowing $\hat{y}$ to get closer to y .